文杰 Homework #1 123090612

#1

(a) Define:
$$\begin{cases} x_{min} = \min(x_x^1, x_x^2, \cdots, x_x^k) \\ x_{max} = \max(x_x^1, x_x^2, \cdots, x_x^k) \\ y_{min} = \min(y_y^1, y_y^2, \cdots, y_y^k) \\ y_{max} = \max(y_y^1, y_y^2, \cdots, y_y^k) \end{cases}$$

Assume that the rectangle is aligned with the x- and y-axes.
The area of the rectangle is given by: $A = (x_{max} - x_{min}) \cdot (y_{max} - y_{min})$

∴ Optimization Problem Formulation:

$$\underset{x_{min}, x_{max}, y_{min}, y_{max}}{\text{minimize}} \quad (x_{max} - x_{min}) \cdot (y_{max} - y_{min})$$

$$\text{subject to:} \quad \min_i x^i \geq x_{min}$$
$$\max_i x^i \leq x_{max}$$
$$\min_i y^i \geq y_{min}$$
$$\max_i y^i \leq y_{max}$$

Classification: non-linear program, convex optimization problem

(b) The perimeter P of the rectangle is given by:

$$P = 2[(x_{max} - x_{min}) + (y_{max} - y_{min})]$$

∴ Optimization Problem Formulation:

$$\underset{x_{min}, x_{max}, y_{min}, y_{max}}{\text{minimize}} \quad 2[(x_{max} - x_{min}) + (y_{max} - y_{min})]$$

$$\text{subject to:} \quad \min_i x^i \geq x_{min}$$
$$\max_i x^i \leq x_{max}$$
$$\min_i y^i \geq y_{min}$$
$$\max_i y^i \leq y_{max}$$

Comparison between the above 2:
The specific choice of the minimum bounding rectangle will be the same in both cases, because the minimum area & perimeter solutions will both use use the extreme x- and y-values of the points.

#2

a) net profit per unit for each product:
product I: $70 - 13 = 57$ product II: $62 - 8 = 54$ product III: $110 - 30 = 80$

Denote the number of units produced of product type I, II, III by x_1, x_2, x_3 respectively.

LP:

$$\underset{x_1, x_2, x_3}{\text{maximize}} \quad Z = 57x_1 + 54x_2 + 80x_3$$

$$\text{subject to:} \quad \frac{1}{3}x_1 + \frac{1}{4}x_2 + \frac{1}{2}x_3 \le 150$$

$$\frac{1}{5}x_1 + \frac{1}{4}x_2 + \frac{1}{4}x_3 \le 80$$

$$x_1, x_2, x_3 \ge 0$$

b) Introduce x_4 for the overtime labor hours.

LP:

$$\underset{x_1, x_2, x_3, x_4}{\text{maximize}} \quad Z = 57x_1 + 54x_2 + 80x_3 - 50\left(\frac{1}{3}x_1 + \frac{1}{4}x_2 + \frac{1}{2}x_3 - 150\right)$$

$$\text{subject to:} \quad \frac{1}{3}x_1 + \frac{1}{4}x_2 + \frac{1}{2}x_3 \le 210$$

$$\frac{1}{5}x_1 + \frac{1}{4}x_2 + \frac{1}{4}x_3 \le 80$$

$$x_1, x_2, x_3 \ge 0$$

c)

```python
import cvxpy as cp

# Decision variables
x1 = cp.Variable(nonneg=True)  # Number of product I
x2 = cp.Variable(nonneg=True)  # Number of product II
x3 = cp.Variable(nonneg=True)  # Number of product III

# Objective function (net profit)
profit = 57*x1 + 54*x2 + 80*x3

# Constraints
assembly_labor_constraint = (1/3)*x1 + (1/4)*x2 + (1/2)*x3 <= 150
testing_labor_constraint = (1/5)*x1 + (1/4)*x2 + (1/4)*x3 <= 80

# Problem definition
problem = cp.Problem(cp.Maximize(profit), [assembly_labor_constraint, testing_labor_constraint])

# Solve the problem
problem.solve()

# Results
print(f"Optimal number of product I: {x1.value:.2f}")
print(f"Optimal number of product II: {x2.value:.2f}")
print(f"Optimal number of product III: {x3.value:.2f}")
print(f"Maximum Net Profit: {problem.value:.2f}")
```

```
✓ import cvxpy as cp ···
...
Optimal number of product I: 0.00
Optimal number of product II: 40.00
Optimal number of product III: 280.00
Maximum Net Profit: 24560.00
```

#3 Let x_{ij} represents the number of rods moved from site i to site j, where $i, j \in \{A, B, C, 1, 2\}$

LP:

$$\underset{x_{ij},\ i,j \in \{A,B,C,1,2\}}{\text{minimize}} \quad 0.1(9x_{A1} + 6x_{A2} + 7x_{B1} + 4x_{B2} + 4x_{C1} + 6x_{C2})$$

$$\text{subject to:} \quad x_{A1} + x_{A2} \le 7000$$

$$x_{B1} + x_{B2} \le 4000$$

$$x_{C1} + x_{C2} \le 3500$$

$$x_{A1} + x_{B1} + x_{C1} \ge 8000$$

$$x_{A2} + x_{B2} + x_{C2} \ge 5500$$

$$x_{ij} \ge 0, \ \forall i, j \in \{A, B, C, 1, 2\}$$

```python
import cvxpy as cp
import numpy as np

# Transportation cost matrix (direct costs)
cost_matrix = np.array([
    [0, 5, 3, 9, 6],   # A
    [5, 0, 9, 7, 4],   # B
    [3, 9, 0, 4, 6],   # C
    [9, 7, 4, 0, 8],   # 1
    [6, 4, 6, 8, 0]    # 2
])

# Floyd-Warshall Algorithm to find the shortest path (indirect costs)
n = len(cost_matrix)
# Using the Floyd-Warshall algorithm to calculate shortest paths
next_site = np.array([[j if i != j else -1 for j in range(n)] for i in range(n)])  # For backtracking the path

# Run the Floyd-Warshall Algorithm to compute shortest paths and backtrack information
for k in range(n):
    for i in range(n):
        for j in range(n):
            if cost_matrix[i, j] > cost_matrix[i, k] + cost_matrix[k, j]:
                cost_matrix[i, j] = cost_matrix[i, k] + cost_matrix[k, j]
                next_site[i, j] = next_site[i, k]

# Decision variables (integer variables)
x_A1 = cp.Variable(integer=True)
x_A2 = cp.Variable(integer=True)
x_B1 = cp.Variable(integer=True)
x_B2 = cp.Variable(integer=True)
x_C1 = cp.Variable(integer=True)
x_C2 = cp.Variable(integer=True)

# Objective function (total cost), using the updated cost matrix
cost = 0.1 * (
    cost_matrix[0, 3] * x_A1 + cost_matrix[0, 4] * x_A2 +  # A -> 1, A -> 2
    cost_matrix[1, 3] * x_B1 + cost_matrix[1, 4] * x_B2 +  # B -> 1, B -> 2
    cost_matrix[2, 3] * x_C1 + cost_matrix[2, 4] * x_C2    # C -> 1, C -> 2
)

# Constraints
constraints = [
    x_A1 + x_A2 <= 7000,  # 7000 rods at A
    x_B1 + x_B2 <= 4000,  # 4000 rods at B
    x_C1 + x_C2 <= 3500,  # 3500 rods at C
    x_A1 + x_B1 + x_C1 >= 8000,  # 8000 rods needed at site 1
    x_A2 + x_B2 + x_C2 >= 5500,  # 5500 rods needed at site 2
    x_A1 >= 0, x_A2 >= 0, x_B1 >= 0, x_B2 >= 0, x_C1 >= 0, x_C2 >= 0  # Non-negativity
]

# Problem definition
problem = cp.Problem(cp.Minimize(cost), constraints)

# Solve the problem
problem.solve()

# Results (modified format)
print("\nOptimal allocation of rods:")
sites = ["A", "B", "C"]
values = [x_A1.value, x_A2.value, x_B1.value, x_B2.value, x_C1.value, x_C2.value]

# Output total cost
print("\nTotal optimal cost:", problem.value)
print(f"x_A1: {x_A1.value}, x_A2: {x_A2.value}")
print(f"x_B1: {x_B1.value}, x_B2: {x_B2.value}")
print(f"x_C1: {x_C1.value}, x_C2: {x_C2.value}\n")

# Function to backtrack and print the path
def get_path(start, end):
    path = []
    while start != end:
        path.append(start)
        start = next_site[start, end]
    path.append(end)
    return path

# Print the rod movements in the specified format, including transit if any
for i in range(3):
    for j in range(2):
        quantity = values[i*2 + j]
        if quantity > 0:
            # Get the path from site i to site j
            path = get_path(i, j + 3)  # 1 and 2 are indexed at 3 and 4
            path_str = " -> ".join([sites[p] if p < 3 else str(p-2) for p in path])
            print(f"Move {quantity} rods from site {sites[i]} to site {j+1} via {path_str}")
```

```
Optimal allocation of rods:

Total optimal cost: 7050.0
x_A1: 4500.0, x_A2: 1500.0
x_B1: -0.0, x_B2: 4000.0
x_C1: 3500.0, x_C2: -0.0

Move 4500.0 rods from site A to site 1 via A -> C -> 1
Move 1500.0 rods from site A to site 2 via A -> 2
Move 4000.0 rods from site B to site 2 via B -> 2
Move 3500.0 rods from site C to site 1 via C -> 1
PS D:\MG\! CUHKSZ\~! 大二 下\MAT3007\Assignments\1>
```

#4 a)

Let X_{ij} be a binary variable:

$$X_{ij} = \begin{cases} 1, & \text{if item } i \text{ is given to daughter } j \\ 0, & \text{if item } i \text{ is not given to daughter } j \end{cases}$$

where: { i refers to the index (from 1 to 12)
j refers to the daughter (B = Bela, D = Daniela, C = Cassandra)

Denote the value of item i by V_i (in 10,000 \$)

"Diamonds" is split into three, while "Jack Russell Race Dogs" is combined into one.

The total values of items assigned to Bela, Daniela and Cassandra are:

$$V_B = \sum_{i=1}^{12} V_i \cdot X_{iB}, \quad V_C = \sum_{i=1}^{12} V_i \cdot X_{ic}, \quad V_D = \sum_{i=1}^{12} V_i \cdot X_{iD}$$

	Item Description	No of Items	Value (per Item, in 10000\$)	Comments
V_1	Sketch by Mondrian	1×	8	
V_2	Bust of Alexander the Great	1×	0.5	
V_3	Yuan Dynasty Chinese Vase	1×	3.5	
V_4	911 Porsche	1×	6	
V_5, V_6, V_7	Diamonds	3×	1.2	
V_8	Louis XV Sofa	1×	0.3	
V_9	Jack Russell Race Dogs	2×	0.3	The will mentions that the dogs can not be separated.
V_{10}	Ancient Sculpture	1×	1	
V_{11}	Sailing Boat	1×	2	
V_{12}	Harley Davidson Motorbike	1×	1	

LP:

$$\underset{X_{iB}, X_{ic}, X_{iD}, \forall i=1,2,\cdots,12}{\text{minimize}} \quad |V_B - V_C|$$

subject to: $X_{iB} + X_{ic} + X_{iD} = 1, \quad \forall i = 1, 2, \cdots, 12$

$$V_D \le V_B \le V_C$$

b) The total value of the legacy is:

$8 + 0.5 + 3.5 + 6 + 1.2 \times 3 + 0.3 + 0.3 \times 2 + 1 + 2 + 1 = 26.5$

$26.5 \div 3 \approx 8.833$ 26.5 is not divisible by 3.

∴ An equal distribution is not possble.

However, an optimal distribution with a minimal difference of 0.1 can be arrived at under the given constraints.

```python
import cvxpy as cp

# Corrected item values (individual diamonds, combined dogs)
values = [
    8.0,   # Sketch
    0.5,   # Bust
    3.5,   # Vase
    6.0,   # Porsche
    1.2,   # Diamond 1
    1.2,   # Diamond 2
    1.2,   # Diamond 3
    0.3,   # Sofa
    0.6,   # Dogs (combined)
    1.0,   # Sculpture
    2.0,   # Boat
    1.0    # Motorbike
]
n_items = len(values)  # Now 12 items
n_daughters = 3

# Binary variables: x[i,j] =1 if item i goes to daughter j
x = cp.Variable((n_items, n_daughters), boolean=True)

# Total values for each daughter
V = [cp.sum(cp.multiply(values, x[:, j])) for j in range(n_daughters)]
V_bela, V_daniela, V_cassandra = V[0], V[1], V[2]

# Constraints
constraints = [cp.sum(x[i, :]) == 1 for i in range(n_items)]  # Each item to one daughter
constraints.append(V_cassandra >= V_daniela)
constraints.append(V_daniela >= V_bela)

# Objective: Minimize Cassandra's value minus Bela's
problem = cp.Problem(cp.Minimize(V_cassandra - V_bela), constraints)
problem.solve(solver=cp.CBC, verbose=True)

# Output results
print(f"Status: {problem.status}")
print(f"Minimal difference (Cassandra - Bela): {problem.value}")
print(f"Bela's total: {V_bela.value}")
print(f"Daniela's total: {V_daniela.value}")
print(f"Cassandra's total: {V_cassandra.value}")

# Print assignments (example output)
for i in range(n_items):
    for j in range(n_daughters):
        if x[i, j].value > 0.5:
            print(f"Item {i+1} ({values[i]}) -> {'Bela' if j==0 else 'Daniela' if j==1 else 'Cassandra'}")
```

```
Status: optimal
Minimal difference (Cassandra - Bela): 0.09999999999999964
Bela's total: 8.8
Daniela's total: 8.8
Cassandra's total: 8.9
Item 1 (8.0) -> Bela
Item 2 (0.5) -> Bela
Item 3 (3.5) -> Cassandra
Item 4 (6.0) -> Daniela
Item 5 (1.2) -> Cassandra
Item 6 (1.2) -> Cassandra
Item 7 (1.2) -> Daniela
Item 8 (0.3) -> Bela
Item 9 (0.6) -> Daniela
Item 10 (1.0) -> Cassandra
Item 11 (2.0) -> Cassandra
Item 12 (1.0) -> Daniela
```

#5

Let $X[i,j]$ be a binary decision variable,

where $X[i,j] = \begin{cases} 1, & \text{if there is a guard placed at the door between room } i \text{ \& room } j \\ 0, & \text{otherwise} \end{cases}$

LP:

$$\underset{X[i,j],\, i,j \in \{A,B,\cdots,J\}}{\text{minimize}} \quad \sum_{i,j \in \{A,B,\cdots,J\}} X[i,j]$$

$$\text{subject to:} \quad \sum_{\text{adjacent rooms } j} X[\min(i,j), \max(i,j)] \geq 1 \quad , \forall i \in \text{rooms}$$

```python
import cvxpy as cp

# Define the rooms and the adjacency relationships based on the given floor plan
rooms = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J']
adjacency = {
    'A': ['B', 'D'],
    'B': ['A', 'C', 'E', 'F'],
    'C': ['B', 'G'],
    'D': ['A', 'E', 'H'],
    'E': ['B', 'H'],
    'F': ['B', 'G'],
    'G': ['C', 'F', 'J'],
    'H': ['D', 'E', 'I'],
    'I': ['H'],
    'J': ['G']
}

# Define the binary decision variable for guards at the doors between rooms
x = {}
for i in rooms:
    for j in adjacency[i]:
        # Ensure each door is represented in only one direction (e.g., A-B or B-A)
        if i < j:
            x[i, j] = cp.Variable(boolean=True)

# Objective function: Minimize the total number of guards
objective = cp.Minimize(sum(x[i, j] for i, j in x))

# Constraints: Each room must be covered by at least one guard
constraints = []
for room in rooms:
    constraints.append(
        sum(x[min(room, adj), max(room, adj)] for adj in adjacency[room]) >= 1
    )

# Define and solve the problem
problem = cp.Problem(objective, constraints)
problem.solve()

# Retrieve the results and display
solution = {}
for i, j in x:
    solution[(i, j)] = x[i, j].value

print("Solution:")
for pair, value in solution.items():
    if value > 0.5:  # Display only the active guard assignments
        print(f"Guard placed at door between rooms {pair[0]} and {pair[1]}")
```

```
Solution:
Guard placed at door between rooms A and D
Guard placed at door between rooms B and E
Guard placed at door between rooms C and G
Guard placed at door between rooms F and G
Guard placed at door between rooms G and J
Guard placed at door between rooms H and I
```